Smart Pantry Manager

Mobile App Development 700 - Practical Assignment (Portfolio of Evidence)

- Name and surname: Kgopotso Mbuli
- Student ITS number: 402309196

- Module: Mobile App Development 700
- Qualification: Bachelor of Information Technology | Year of study: 3rd Year Semester: second semester
- Date: 23 October 2026
- GitHub repository: https://github.com/Kgopotso24/SmartPantry

Table of Contents

1. Introduction
2. System Design
3. Screenshots of Every Output
4. Key Code Snippets
5. Challenges and Solutions
6. Conclusion and Reflection
7. Reference List

1. Introduction

1.1 The problem

Many households throw away food because they do not know what to cook with the ingredients they already have, or because items expire before they are used. Shopping for missing ingredients also costs time and money.

1.2 The solution

Smart Pantry Manager is an Android application, written in Java in Android Studio, that helps users cut food waste. The user records the ingredients they actually have at home (their pantry). The app then suggests recipes they can cook using only those ingredients,

so no shopping trip is needed. Its core rule is strict matching: a recipe is only called ready when every ingredient it needs is in the pantry in at least the required quantity.

1.3 Who it is for

The app is aimed at students, families and anyone who wants to cook from what is already in the kitchen, and who wants to use food before it expires.

1.4 Overview of the app

- Pantry management: add, edit and delete ingredients with a quantity, unit and optional expiry date.
- A Home dashboard showing the number of ingredients, items to use soon, a freshness bar, a recipe to cook tonight and recipes that are almost ready.
- A Suggestions screen that lists recipes the pantry fully covers, lists recipes that are one or two ingredients short in a separate section, and lets the user ask for a specific meal such as pizza. For a requested meal, the app shows what is missing and suggests swaps, preferring ingredients the user already has.
- 25 built-in recipes, plus the ability for users to write and save their own recipes.
- A Settings screen with the user's name, an expiring-soon alert switch, a default unit, and buttons to load or clear sample data.

The app is written in Java using Android Studio, stores its data in a local SQLite database, and was tested on a physical Android phone. No maps, GPS or location features are used, as required by the brief.

1.5 Database choice

I chose SQLite because the app's data (a personal pantry and recipes) belongs on the device. SQLite needs no internet connection, account or server, it persists data when the app is closed, and it is the approach taught in the persistent-data chapter of the module. A cloud database would add network and login complexity that this app does not need.

1.6 How the app meets the brief

Requirement	How the app meets it
Add, edit and delete pantry items	AddEditActivity and DatabaseHelper; delete asks for confirmation

Requirement	How the app meets it
Pantry list in a RecyclerView	MainActivity with a custom PantryAdapter
15-20 recipes seeded on first run	18 recipes seeded on first run, plus 7 added in database version 3 (25 in total)
Suggested recipes with strict matching	SuggestionsActivity ('Cook with what you have') using IngredientMatcher
Recipe detail screen	RecipeDetailActivity
Settings screen	SettingsActivity (name, expiring alerts, default unit)
Message when nothing matches	'No recipes match your pantry yet - add more ingredients' on the Can make now list
Matching robust to plurals and units	Names are normalised (tomato = tomatoes) and units converted (kg to g, l to ml)
Four or more screens	Eight screens
Intents to move between screens and pass data	Intents with extras for item and recipe ids
Navigation element	Bottom navigation bar (Home, Pantry, Suggest, Settings)
Input validation	Add ingredient and New recipe forms
Full CRUD with persistence	SQLite; data is still there after closing and reopening
Optional 'Almost There' list	Shown separately from the strict list on Home and Suggestions
No maps, GPS or location	None used

2. System Design

2.1 Screens

Screen	Activity	Purpose
Home	HomeActivity	Dashboard: greeting, stat tiles, freshness bar, quick actions, tonight's pick, almost-there list
Pantry	MainActivity	Lists all ingredients (Read) with Edit and Delete
Add / Edit Ingredient	AddEditActivity	Create and Update an ingredient, with validation
Suggestions	SuggestionsActivity	Ready recipes, almost-there recipes, and meal requests with swaps
All Recipes	SuggestedActivity	Browse every recipe: Can make now or All recipes
Recipe Detail	RecipeDetailActivity	Full ingredients and method; delete your own recipes
New Recipe	AddRecipeActivity	Write a recipe with a dynamic list of ingredient rows
Settings	SettingsActivity	Name, expiring alerts, default unit, sample data

2.2 Screen flow

The bottom navigation bar moves between Home, Pantry, Suggest and Settings. Intents move the user to the other screens and pass ids as extras.

- Home opens Pantry, Suggestions, Add Ingredient, New Recipe, Recipe Detail (tonight's pick) and All Recipes (almost-there card).
- Pantry opens Add / Edit Ingredient.
- Suggestions opens Recipe Detail, All Recipes and New Recipe.
- All Recipes opens Recipe Detail.
- Recipe Detail can delete a recipe the user created.

2.3 Data model

Data is stored in a local SQLite database (smart_pantry.db, version 3) managed by a SQLiteOpenHelper called DatabaseHelper.

pantry

Column	Type	Notes
id	INTEGER	Primary key, autoincrement
name	TEXT	Required
quantity	REAL	Required, greater than 0
unit	TEXT	g, kg, ml, l, cup, tbsp, tsp or pcs
expiry	TEXT	Optional, format yyyy-MM-dd

recipes

Column	Type	Notes
id	INTEGER	Primary key, autoincrement
name	TEXT	Required
steps	TEXT	Preparation method
is_custom	INTEGER	0 for built-in recipes, 1 for recipes the user wrote

recipe_ingredients

Column	Type	Notes
id	INTEGER	Primary key, autoincrement
recipe_id	INTEGER	Foreign key to recipes.id, ON DELETE CASCADE
name	TEXT	Ingredient name
quantity	REAL	Amount required
unit	TEXT	Unit of the amount

One recipe has many recipe ingredients. The pantry table is separate: matching compares ingredient names and quantities in code rather than through a foreign key. The user's name, the alerts switch and the default unit are stored in SharedPreferences.

2.4 The strict-matching rule

The rule is implemented in IngredientMatcher. A recipe is suggested only when every ingredient is covered by the pantry in at least the required quantity. To cope with real-world messiness, the matcher does three things:

1. It normalises names to lower case and singular form, so Tomatoes matches tomato.
2. It converts units to a base unit: kg to g, and l, cup, tbsp and tsp to ml.
3. It adds up the same ingredient when it appears in several pantry rows.

A recipe that needs five ingredients when the pantry has four is not suggested. Recipes that are one or two ingredients short are shown only in a separate 'Almost there' section and are never mixed into the strict list.

3. Screenshots of Every Output

Insert a real screenshot from the running app for each figure below, with the caption underneath.

Figure 1: Home screen with an empty pantry. The dashboard shows zero ingredients and a message that nothing is ready to cook.

[Insert screenshot]

Figure 2: Empty Pantry screen. A friendly message appears instead of a blank screen.

[Insert screenshot]

Figure 3: Add Ingredient form. The Create step, with name suggestions, quantity, unit and an optional expiry date.

[Insert screenshot]

Figure 4: Validation errors. Saving with an empty name or empty quantity shows error messages and does not save.

[Insert screenshot]

Figure 5: More validation. A quantity of 0 and a name containing numbers are both rejected.

[Insert screenshot]

Figure 6: Pantry list. The Read step: ingredients in a RecyclerView with quantity chips and expiry text.

[Insert screenshot]

Figure 7: Editing an ingredient. The Update step: the form opens pre-filled with the saved values.

[Insert screenshot]

Figure 8: Deleting an ingredient. The Delete step: a confirmation dialog appears before the item is removed.

[Insert screenshot]

Figure 9: Home dashboard with the sample pantry. Stat tiles, freshness bar, tonight's pick and the almost-there list.

[Insert screenshot]

Figure 10: Suggestions - Cook with what you have. Only recipes whose ingredients are all in the pantry are listed as ready.

[Insert screenshot]

Figure 11: Strict matching in action. A recipe is absent while one ingredient is missing and appears once that ingredient is added (use two screenshots, before and after).

[Insert screenshot]

Figure 12: Requesting a meal. Asking for pizza shows what is missing and a swap the user already has.

[Insert screenshot]

Figure 13: Requesting an unknown meal. The app offers to create the recipe and suggests something the user can cook now.

[Insert screenshot]

Figure 14: All Recipes. The Can make now and All recipes views, with missing ingredients shown.

[Insert screenshot]

Figure 15: Recipe Detail. The full ingredient list and method for a selected recipe.

[Insert screenshot]

Figure 16: New Recipe. The form with dynamic ingredient rows, and the saved recipe appearing in the list.

[Insert screenshot]

Figure 17: Settings. Name, expiring-soon alerts, default unit and the sample data buttons.

[Insert screenshot]

Figure 18: Data persists. The pantry is still present after the app is closed and reopened.

[Insert screenshot]

4. Key Code Snippets

4.1 Strict matching (IngredientMatcher.missing)

```
for (RecipeIngredient needed : recipe.getIngredients()) {
    double[] have = stock.get(normalise(needed.getName()));
    boolean covered;
    if (have == null) {
        covered = false;                        // not in the pantry at all
    } else {
        int cat = category(needed.getUnit());
        double required = toBase(needed.getQuantity(), needed.getUnit());
        if (have[cat] > 0) {
            covered = have[cat] + 0.0001 >= required;   // enough quantity?
        } else {
            covered = have[WEIGHT] + have[VOLUME] + have[COUNT] > 0;
        }
    }
    if (!covered) {
        missing.add(needed.getName());
    }
}
```

This loop is the heart of the app. It checks every ingredient of a recipe against the pantry totals and collects the ones that are not covered. A recipe is ready only when the list stays empty. It was written this way so that one function serves the strict list (nothing missing), the almost-there list (one or two missing) and the missing-ingredient messages.

4.2 Handling plurals (IngredientMatcher.singular)

```
private static String singular(String w) {
    if (w.length() <= 3) return w;
    if (w.length() > 4 && w.endsWith("ies")) {
        return w.substring(0, w.length() - 3) + "y";      // berries -> berry
    }
    if (w.endsWith("oes") || w.endsWith("ches") || w.endsWith("shes")
            || w.endsWith("xes") || w.endsWith("sses")) {
        return w.substring(0, w.length() - 2);            // tomatoes ->
tomato
    }
    if (w.endsWith("s") && !w.endsWith("ss") && !w.endsWith("us")) {
        return w.substring(0, w.length() - 1);            // eggs -> egg
    }
    return w;
}
```

The brief warns that an exact string match will be marked down. This small set of rules turns plural names into singular form so that tomato and tomatoes are treated as the same ingredient, without needing a full language-processing library.

4.3 Update and Delete in SQLite (DatabaseHelper)

```
public int updateItem(int id, String name, double quantity, String unit,
String expiry) {
    ContentValues values = new ContentValues();
    values.put("name", name);
    values.put("quantity", quantity);
    values.put("unit", unit);
    values.put("expiry", expiry);
    return getWritableDatabase().update("pantry", values, "id = ?",
            new String[]{String.valueOf(id)});
}

public int deleteItem(int id) {
    return getWritableDatabase().delete("pantry", "id = ?",
            new String[]{String.valueOf(id)});
}
```

These methods complete the CRUD cycle. ContentValues holds the new column values, and the question mark in the where clause is filled by the argument array, which avoids building SQL by hand and keeps the query safe. The data is written to the database file, so it is still there after the app is closed.

4.4 RecyclerView adapter and Intents (PantryAdapter and MainActivity)

```
// PantryAdapter: binding one ingredient to its card
holder.tvName.setText(item.getName());
holder.tvQty.setText(IngredientMatcher.formatQty(item.getQuantity()) + " " +
item.getUnit());
holder.btnEdit.setOnClickListener(v -> listener.onEdit(item));
holder.btnDelete.setOnClickListener(v -> listener.onDelete(item));

// MainActivity: Intent that opens the edit screen and passes the item id
Intent intent = new Intent(MainActivity.this, AddEditActivity.class);
intent.putExtra(AddEditActivity.EXTRA_ITEM_ID, item.getId());
startActivity(intent);
```

The custom adapter turns each row of the database into a card in the RecyclerView. The adapter does not decide what Edit means: it calls a listener, and MainActivity answers with an Intent that carries the item id to AddEditActivity, which loads that record and switches to edit mode.

4.5 Finding swaps for missing ingredients (SuggestionEngine.analyse)

```
List<Substitutions.Alt> options = Substitutions.optionsFor(key);
Substitutions.Alt inPantry = null;
search:
for (Substitutions.Alt alt : options) {
    String altKey = IngredientMatcher.normalise(alt.name);
    for (PantryItem item : pantry) {
        if (IngredientMatcher.normalise(item.getName()).equals(altKey)) {
            inPantry = alt;      // a swap the user already has
            break search;
        }
    }
}
```

When a requested recipe has a missing ingredient, the app looks up known swaps in a built-in table and checks whether the user already owns one of them. This works offline and keeps the strict rule intact, because swaps are only shown in the requested-meal and almost-there sections.

5. Challenges and Solutions

5.1 The first push to GitHub was rejected

My first attempt to push the project failed with an authentication error saying that password authentication is not supported. I solved it by creating a personal access token on GitHub with the repo permission, clearing the stored credentials, and entering the token instead of my account password when Git asked for it. After that the commits were pushed successfully.

5.2 The package name did not match the template

My project is named SmartPantrySystem, so Android Studio generated the package com.example.smartpantrysystem, which did not match the package name in the code I was adding. This would have caused unresolved references to the generated R class. I fixed it by reading the namespace from the module's build file and updating the package line of each Java file to match it.

5.3 Matching had to cope with messy real-world input

A naive text comparison failed for simple differences such as tomato and tomatoes, or kilograms and grams, which the brief says will be marked down. I solved it by normalising ingredient names to lower-case singular form, converting units to a base unit before comparing, and adding up the same ingredient across several pantry rows. I tested the logic on its own with a set of cases, including one that checks that four of five ingredients is not enough.

6. Conclusion and Reflection

6.1 What I learned

Building Smart Pantry Manager taught me how the main parts of an Android app fit together: Activities and their lifecycle, layouts, Intents for passing data, RecyclerView with a custom adapter, and persistent storage with SQLite. I also learned to put the important rule of the app, strict matching, in one separate class so that it could be tested on its own and reused by several screens. Working with Git and GitHub throughout showed me the value of small, descriptive commits.

6.2 What I would improve with more time

- Add a button to mark a recipe as cooked, which would subtract the used quantities from the pantry.
- Add recipe photos and servings, and scale quantities for different numbers of people.
- Send real notifications for items about to expire.
- Convert between weight and volume using ingredient densities, so that grams and teaspoons could be compared exactly.
- Add a larger swap table and more built-in recipes.
- Sync the pantry to the cloud so it can be shared between devices.

7. Reference List

Android Developers (n.d.) *Save data using SQLite*. Available at: https://developer.android.com/training/data-storage/sqlite (Accessed: [date]).

Android Developers (n.d.) *Create dynamic lists with RecyclerView*. Available at: https://developer.android.com/develop/ui/views/layout/recyclerview (Accessed: [date]).

Android Developers (n.d.) *Intents and intent filters*. Available at: https://developer.android.com/guide/components/intents-filters (Accessed: [date]).

Android Developers (n.d.) *The activity lifecycle*. Available at: https://developer.android.com/guide/components/activities/activity-lifecycle (Accessed: [date]).

Android Developers (n.d.) *Save key-value data*. Available at: https://developer.android.com/training/data-storage/shared-preferences (Accessed: [date]).

GitHub Docs (n.d.) *Managing your personal access tokens*. Available at: https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens (Accessed: [date]).

[Add any other tutorials, documentation or tools you used, and acknowledge any AI assistance if your institution's policy requires it.]